APPENDIX OUTLINE

A. Omitted Proofs . . . 2
A.1 Hardness and Scope Decomposition . . . 2
A.2 Exact Planning for Small Scopes . . . 2
A.3 Coordinated Request Grouping . . . 2
A.4 Global Prefix Index . . . 4
A.5 Prefix-Ready Admission . . . 5

B. DP for Small Scopes . . . 5

C. Additional Experiment . . . 5
C.1 Sensitivity to the Number of Evidence Items . . . 5
(RQ13) Do the evidence-count trends hold across all four datasets? . . . 5
C.2 Planner and Workload Sensitivity . . . 6
(RQ14) How many orders does the index distinguish as requests retrieve more evidence? . . . 6
(RQ15) How many rounds does regrouping need? . . . 6
(RQ16) How does the density of evidence sharing affect the lead of KaleidoKV? . . . 6

D. Experimental Setup . . . 7
Datasets and Workload Construction . . . 7
Baseline Configurations . . . 7
Evaluation Metrics . . . 8
Models and Environment . . . 8
Implementation and Runtime Settings . . . 8

A OMITTED PROOFS

We organize proofs by component and restate each result with its original number. The weight W follows Definition 2: each distinct evidence-item prefix contributes its last item's token length once.

A.1 Hardness and Scope Decomposition

The first two results establish the difficulty of joint layout planning and justify splitting the workload into independent reuse scopes (Section 4.1). A reuse scope is a connected component of requests linked by shared evidence, as in Definition 5. We write $\mathcal{E}(S)$ for the union of all evidence items retrieved by the requests in scope S.

Theorem 1 (Hardness). *Choosing the evidence orders of all requests so that their prefix tree has the smallest weight is NP-hard.*

Proof of Theorem 1. We reduce from minimum vertex cover [5]. Given a simple graph $H = (V, F)$ without isolated vertices, we create one evidence item of one token for each vertex, and one request for each edge $\{u, v\}$ with evidence $\{u, v\}$ and no dependencies. A layout then chooses which endpoint of each edge comes first. Let A be the set of items placed first. The tree has one node for each item of A, and each request adds one node for its second item, since no two edges have the same endpoints. Hence $W = |F| + |A|$. The set A contains an endpoint of every edge, so it is a vertex cover. Conversely, placing an endpoint in a vertex cover C first gives a layout with $A \subseteq C$. Therefore, the minimum weight is $|F|$ plus the size of a minimum vertex cover, and the reduction is polynomial. The problem is thus NP-hard even with two items per request and no dependencies. If H is connected, all requests form one scope, so the problem stays NP-hard within a single scope. Vertex cover remains NP-hard on connected graphs, since joining a new vertex z to one vertex of each component and to a new leaf raises the minimum cover by exactly one, with z in the cover. □

Lemma 1 (Scope decomposition). *Given the request set $\mathcal{R}$ and its scope set $\mathcal{S}$, placing a request of a scope $S \in \mathcal{S}$ with the requests of another scope $S' \in \mathcal{S}$ can never lower the prefix tree weight of $\mathcal{R}$.*

Proof of Lemma 1. By Definition 5, scopes are connected components under shared evidence, giving the partition and properties (i) and (ii). Take a request r of a scope S and a request r' of another scope S'. In every layout, their first items lie in $\mathcal{E}(S)$ and $\mathcal{E}(S')$, which are disjoint by (ii), so their paths separate at the root. Requests of different scopes thus never share a node, and placing r with the requests of S' cannot lower W. The total weight is a sum of scope weights, each determined only by its own requests' orders. Thus, the scope minima sum to the global minimum of W. □

A.2 Exact Planning for Small Scopes

The dynamic program in Appendix B optimizes nested request groups when evidence items have no ordering dependencies. For a group G, let $I(G)$ be its shared evidence and $L(G) = \sum_{e \in I(G)} \ell(e)$ its total token length. We relate recurrence savings to tree weight.

Proposition 4 (Optimality of DP). *For a scope with d distinct evidence sets and no dependencies, DP finds a layout of minimum weight. Its recurrence takes $O(4^d)$ time after the intersection lengths and request multiplicities of candidate groups are precomputed.*

Proof of Proposition 4. Without dependencies, every order is legal. Here, we consider only groups with at least two requests, since singleton groups contribute zero saving. These groups are nested or disjoint. For a group G, let U_G be the smallest group that strictly contains G, or the virtual root $\perp$ otherwise, with $L(\perp) = L_O(\perp) = 0$.

Weight in terms of groups. Let Λ be the total length of the evidence of all requests. Every request passes one node per item, so $W = \Lambda - \sum_v (n_v - 1)\,\ell(e_v)$, where n_v is the number of requests on node v. For a layout O, let $L_O(G)$ be the length of the path from the root to the deepest node whose requests are exactly G. The nodes whose requests are exactly G form a contiguous path directly below the node of U_G, with all items in $I(G)$. Hence $L_O(G) \le L(G)$ and

$$W = \Lambda - \sum_G \big(|G| - 1\big)\big(L_O(G) - L_O(U_G)\big). \tag{5}$$

No layout beats DP. Some layout of minimum weight gives the same order to requests with the same evidence set: giving all of them one of their current orders adds no node and cannot raise W. The groups of this layout are thus unions of such classes. In the saving term of Eq. (5), the length $L_O(H)$ of a group H has the factor $(|H| - 1) - \sum_{G:\, U_G = H} (|G| - 1)$, which is nonnegative since these G are disjoint subsets of H. Therefore, replacing every $L_O(H)$ by the larger $L(H)$ gives $W \ge \Lambda - \sum_G (|G| - 1)(L(G) - L(U_G))$. Groups with $L(G) = L(U_G)$ add nothing and can be dropped, and the rest is one of the choices in Eq. (7). This equation maximizes over all such choices, since the choices inside disjoint groups are independent. Hence every legal layout has weight at least $\Lambda - \mathit{Save}(\perp)$.

DP attains this bound. Each request places the items of its groups chosen by DP from the largest group to the smallest, with one fixed item order within each block, and then its remaining items. The members of each chosen group G then share the items of $I(G) \setminus I(U_G)$ on nodes that all of them pass. Nested groups have disjoint blocks, so the groups that share items on one node v are disjoint, and their values of $|G| - 1$ add up to at most $n_v - 1$. Thus, this layout attains the saving bound $\mathit{Save}(\perp)$ and is optimal.

Running time. After precomputing intersection lengths and request multiplicities, DP works on the d classes, so there are 2^d candidate groups. For a fixed U containing j classes, a second recurrence over subsets X of these classes either leaves the first class of X outside every group or puts it into a group $G \subseteq X$. It examines at most 3^j pairs $G \subseteq X \subseteq U$. Summing over real groups gives $\sum_j \binom{d}{j} 3^j = 4^d$; the virtual root adds $O(3^d)$ time. □

A.3 Coordinated Request Grouping

For large scopes, the planner moves requests together to revise shared prefixes. We first characterize when a joint move gains more

than separate moves, and then bound planning cost when orders are enumerated and each change rebuilds the full prefix tree.

Lemma 2 (Sources of joint gain). *For a change that moves a set M of requests together, let $\Delta W(M)$ be its reduction of W, and let $\Delta W(r)$ be the reduction when $r \in M$ moves alone to the same order. Then $\Delta W(M) > \sum_{r\in M} \Delta W(r)$ only if the change (i) creates a prefix that no request used before and at least two requests of M share, or (ii) empties a prefix that at least two requests shared before.*

Proof of Lemma 2. We compare the joint move with the single moves of its requests, all measured from the same layout. By Definition 2, the weight adds the length of every prefix in use, so it suffices to compare them on one prefix t. If t is in use, the joint move saves $\ell(e_t)$ on t only if it empties t. A single move empties t only if t has one request, and it then saves as much. The joint move thus gains more only if t has at least two requests and all of them leave, which is case (ii). If t is not in use, the joint move pays $\ell(e_t)$ once, while the single moves pay it once for each of the b moved requests that use t afterwards. The joint move thus saves $(b-1)\,\ell(e_t)$ more, which is positive only if $b \geq 2$. This is case (i). On all other prefixes, joint gain is bounded by the sum of individual gains, which proves the claim after summing over the tree. □

Lemma 3 (Gain of a merge). *Let t be a sequence of distinct evidence items such that no request uses any nonempty prefix of t yet, and let a merge move a set $M \subseteq L(t)$ of requests onto t. If every non-singleton group that loses requests keeps other members and the requests of M share only the prefix t, the merge lowers W by*

$$\Delta W = \sum_{r\in M} \big(\ell(t) - u_r\big) - \ell(t) = \sigma(G_t) - \sum_{r\in M} u_r, \tag{2}$$

where $\sigma(G_t) = (|M|-1)\,\ell(t)$ is the saving of the new group G_t on t. Hence the merge lowers W if and only if G_t saves more than the requests of M reuse in the current layout, which requires at least two of them to reuse fewer than $\ell(t)$ tokens in the layout before the merge.

Proof of Lemma 3. Every previously shared prefix remains used outside M, so moving M removes $\sum_{r\in M} u_r$ from the existing savings. The new paths share only the previously unused prefix t, adding $(|M|-1)\ell(t)$ to the savings. The net reduction is

$$\Delta W = (|M|-1)\ell(t) - \sum_{r\in M} u_r = \sigma(G_t) - \sum_{r\in M} u_r.$$

If fewer than two members satisfy $u_r < \ell(t)$, then $\sum_{r\in M} u_r \geq (|M|-1)\ell(t)$, since reuse is nonnegative. Hence a positive gain requires at least two members whose reuse is below $\ell(t)$. □

Lemma 4 (Gain of a dissolve). *Let no request outside a group G use any nonempty prefix of p_G, and no two members of G share evidence items after p_G. While another member stays in G, moving a member r out lowers W by $u'_r - \ell(p_G)$, and dissolving G lowers W by*

$$\Delta W = \ell(p_G) - \sum_{r\in G} \big(\ell(p_G) - u'_r\big) = \sum_{r\in G} u'_r - \sigma(G), \tag{3}$$

where $\sigma(G) = (|G|-1)\,\ell(p_G)$. Hence G is premature if and only if its members reuse more in other groups than G currently saves.

Proof of Lemma 4. Let $T_r = \sum_{e\in\mathcal{E}(r)} \ell(e)$. By assumption, no request outside G uses any nonempty prefix of p_G, and the members of G share no evidence items after p_G. Their old paths therefore consist of the shared prefix p_G and mutually private suffixes. Every new path in the dissolve avoids the first node of G, so earlier placements cannot use the old shared prefix or any of its descendants.

Consider a member r that moves while another member remains in G. Removing r deletes its private suffix, of weight $T_r - \ell(p_G)$, while p_G remains in use. Placing r adds $T_r - u'_r$ tokens, where u'_r is measured immediately after this placement and includes reuse of earlier placements. Therefore, the reduction in W for this move is

$$(T_r - \ell(p_G)) - (T_r - u'_r) = u'_r - \ell(p_G).$$

The last departure also removes p_G, contributing an additional reduction of $\ell(p_G)$. Summing the successive reductions gives

$$\begin{aligned}\Delta W &= \ell(p_G) - \sum_{r\in G} \big(\ell(p_G) - u'_r\big) \\ &= \sum_{r\in G} u'_r - (|G|-1)\ell(p_G) \\ &= \sum_{r\in G} u'_r - \sigma(G).\end{aligned}$$

Hence the completed dissolve lowers W exactly when the members' reuse summed over their placements exceeds the saving of G. □

Theorem 2 (Planning cost). *Let the workload hold N requests, each retrieving at most k evidence items, and let merges and dissolves each run at most $I_{\max}$ rounds. If candidate orders are enumerated explicitly and every change is evaluated by recomputing W, the scope split, the greedy start, and the merges and dissolves take $O(I_{\max} \cdot k! \cdot k^2 \cdot N^2)$ expected time in total, apart from the ILP solves.*

Proof of Theorem 2. With hash tables, recomputing W takes expected $O(kN)$ time, since it inserts at most k items for each of the N requests. The scope split takes $O(kN)$ time. The greedy start takes $O(k^2 N \log N)$ time with a priority queue, since each request enters at most $k+1$ recursive calls. Evaluating one ILP solution per small scope by recomputation takes $O(kN^2)$ time in total; the ILP solves are excluded. Building the candidate lists and sorting them once takes $O(k2^k N + 2^k N \log N)$ time before regrouping starts.

In a merge round, each candidate X tries $L(X)$ and its partition by first item, giving at most $2|L(X)|$ tries. Each try enumerates $|X|!$ prefix orders, preserving the remaining order of each member. The total number of candidate evaluations in the round is at most

$$\begin{aligned}\sum_X 2|L(X)|\,|X|! &\leq 2\sum_r \sum_{\emptyset\neq X\subseteq\mathcal{E}(r)} |X|! \\ &\leq 2N \sum_{j=1}^{k} \binom{k}{j} j! < 2eNk!.\end{aligned}$$

Each evaluation recomputes W in $O(kN)$ time, so a merge round takes $O(k!\,kN^2)$ time. This bound also covers the single-request improvements performed at the end of the round for each request.

In a dissolve round, each group present at the start is tried once, and groups that have gained members are skipped. Each request is placed at most k times, once per group on its initial path. Each of the at most kN placements tries at most $k!$ orders, each with one

$O(kN)$ recomputation, giving $O(k!\,k^2N^2)$ time per round. With at most $I_{\max}$ rounds of each operation, all initialization costs are absorbed into $O(I_{\max} \cdot k! \cdot k^2 \cdot N^2)$, apart from the ILP solves. □

A.4 Global Prefix Index

The index evaluates changes through reference counts and searches for orders through reusable evidence sets (Section 4.2). A node is visible when a request outside the hidden move group still uses it. The entry $Sets[X]$ contains visible nodes whose root paths hold exactly the items of X. The following results justify evaluation and order selection, then bound indexed planning over all rounds.

Lemma 5 (Evaluation by reference counts). *For a change that gives some requests new orders, let V^- be the nodes that lose their last request and V^+ the nodes that gain their first one. Then*

$$\Delta W = \sum_{v \in V^-} \ell(e_v) - \sum_{v \in V^+} \ell(e_v),$$

where both sets lie on the old and new paths of these requests.

Proof of Lemma 5. A node v adds $\ell(e_v)$ to W exactly when its count is positive. A change alters the counts only on the old and new paths of the moved requests, and a node changes W only when its count crosses zero. A node that loses its last request lies in V^- and removes $\ell(e_v)$ from W. A node that gains its first request lies in V^+ and adds $\ell(e_v)$. Summing these changes gives Eq. (4). □

For a merge onto an evidence set X, $L(X)$ contains the requests that retrieve all items of X, and $M \subseteq L(X)$ is the move group. Removing X from the current orders of M gives the residual tree U_M. We write $U_M \cap \mathcal{T}_v$ for the residual nodes visible below v.

Lemma 6 (Best merge order). *With M hidden, the order of X that gives the smallest W can be evaluated in the following two cases. (i) If $Sets[X]$ is empty, no visible prefix consists of exactly the items in X, and no node of U_M can be reused. The best order follows a node of $Sets[Y]$, $Y \subseteq X$, with the largest $\ell(Y)$, and then takes the other items of X in any order. This case always holds for $M = L(X)$. (ii) If $Sets[X]$ is not empty, the best order reuses all of X. It is the path of a node $v \in Sets[X]$ with the largest $W(U_M \cap \mathcal{T}_v)$, the weight of the nodes of U_M that are visible below v with matching item sequences.*

Proof of Lemma 6. Let W' be the weight of the visible nodes with M hidden, which does not depend on t. The new paths consist of the prefixes of t, of total weight $\ell(X)$, followed by the nodes of U_M, of total weight $W(U_M)$. The weight after the move is W' plus the weight of the new nodes that are not visible. A request that passes a node also passes its parent, so every node below a node that is not visible is also not visible. Let $Y(t)$ be the items on the longest visible prefix of t, with M hidden in the set table.

Case (i). Since $Sets[X]$ is empty, $Y(t) \neq X$, so the last node of t and all nodes of U_M below it are not visible. The weight after the move is thus $W' + \ell(X) - \ell(Y(t)) + W(U_M)$, which depends on t only through $\ell(Y(t))$. Every $Y(t)$ has a node in $Sets[Y(t)]$ (including the root for the empty set), so $\ell(Y(t))$ is at most the largest $\ell(Y)$ over $Y \subseteq X$ with $Sets[Y]$ not empty, and the order of the lemma reaches this value. If $M = L(X)$, any request using a node that holds all of X belongs to M. Hiding M removes every such node from the set table, so case (i) always applies when the entire list $L(X)$ is moved.

Case (ii). If t is the path of a node $v \in Sets[X]$, all of t is visible, and the weight after the move is $W' + W(U_M) - W(U_M \cap \mathcal{T}_v) \leq W' + W(U_M)$. Any order not represented in $Sets[X]$ has $Y(t) \neq X$ and gives $W' + \ell(X) - \ell(Y(t)) + W(U_M) > W' + W(U_M)$, as in case (i), because every evidence item has positive length. Therefore, the best order is the path of the node v with the largest overlap. □

Lemma 7 (Equivalent orders). *When a request is placed, two orders that reuse the same items have the same ΔW, and ΔW grows with the total length of these items. The best order reuses the longest set of items, and a request with k items has at most 2^k such sets.*

Proof of Lemma 7. Remove r from its old path first, which gives counts n'_v and a weight W' that do not depend on the new order. A request that passes a node also passes its parent, so every node below a node with $n'_v = 0$ also has a zero count. An order o of r thus follows nodes with positive counts for its first items, which form the set $S(o)$ of evidence items that it reuses, and nodes with zero counts for all its other items. Placing r on o adds exactly these other items to W', so $\Delta W = W - W' - \sum_{e \in \mathcal{E}(r) \setminus S(o)} \ell(e)$. This depends on o only through $S(o)$ and increases with $\ell(S(o))$. Since $S(o)$ is a subset of at most k items, there are at most 2^k classes of orders with the same gain for the individual request being placed.

During a dissolve, the same argument applies to the temporary tree with G hidden and earlier placements inserted. Let g be the forbidden first node of G and p its parent. A visible prefix can fail to have a completion avoiding g only if it ends at p and e_g is the only remaining item. Thus at most one node of any $Sets[Y]$ is infeasible, and reading two nodes suffices. Choose a feasible visible prefix of maximum total length. If items remain, the first step of a completion avoiding g must be unused; otherwise, its visible child would give a longer feasible prefix. If no items remain, no further step is needed. Thus, placement finds a best feasible order without enumerating the member's $k!$ evidence permutations, with other orders fixed. □

Theorem 3 (Planning cost with the index). *For N requests with at most k evidence items each and at most $I_{\max}$ rounds each of merges and dissolves, indexed planning takes $O(I_{\max} \cdot k \cdot 3^k \cdot N + 2^k \cdot N \log N)$ expected time, excluding the greedy start and the ILP solves.*

Proof of Theorem 3. With hashing, a table access takes expected constant time and forming an item-set key takes $O(k)$ time. We include both costs in the expected-time bounds below.

Basic operations. By Lemma 5, an evaluation reads at most $2k$ nodes per moved request. Applying a change, hiding a request, or restoring it updates the same nodes and their set-table entries, taking $O(k^2)$ time per request including set-key formation. Placing a request takes at most 2^k lookups, each reading at most two nodes, and $O(k)$ further steps to form the order, which gives $O(k2^k)$ time.

Candidates. A request has fewer than 2^k nonempty evidence subsets, so there are $Q < 2^kN$ candidate sets, which take $O(k2^kN)$ time to build. Sorting them once by priority takes $O(Q \log Q) = O(k2^kN + 2^kN \log N)$ time, and the order is reused in each round.

Merge rounds. A round tries each list $L(X)$ as a whole and through its partition, so the tries of X hold at most $2|L(X)|$ member occurrences and number at most $2|L(X)|$. Each request lies in fewer than 2^k lists, so a round has $O(2^kN)$ member occurrences, and hiding

them, building U_M, evaluating, and restoring take $O(k^2 2^k N)$ time, including set-key formation and new-path construction. In case (i) of Lemma 6, a try on X takes at most $2^{|X|}$ lookups. Over a round, the lookups number at most $\sum_X 2|L(X)|2^{|X|} \le 2\sum_r \sum_{X\subseteq\mathcal{E}(r)} 2^{|X|} \le 2 \cdot 3^k N$, which takes $O(k3^k N)$ time. In case (ii), the search visits only visible nodes, and each visit tests the matching edge of U_M in constant time. At most kN nodes are visible when a round starts, and each accepted member occurrence adds at most k more, so $O(k2^k N)$ nodes are visible during a round. A node x is visited for X only if x or one of its ancestors holds exactly X, which leaves at most k sets X. For each of them, at most $|X|$ parts begin with an item of X, and only one part begins with the item that follows X on the path of x. Each pair of a set and a part is tried once per round, so x is visited $O(k^2)$ times. The searches thus take $O(k^3 2^k N)$ time, which is $O(k3^k N)$ because $k^2 2^k = O(3^k)$. The single-request improvements of a round take $O(k2^k N)$ time across the batch.

Dissolve rounds. A request lies in at most k groups, since its path has at most k nodes. A round tries each group present at its start once and skips groups that have gained members. It places at most kN members, which takes $O(k^2 2^k N) = O(k3^k N)$ time. Hiding, restoring, evaluating, and new-path key construction add at most $O(k^2)$ time per placement, also within this bound for the round.

Total. The scope split takes $O(kN)$ time, and evaluating the ILP solutions takes $O(k)$ time per request. Index construction, sorting, and at most $I_{\max}$ merge and dissolve rounds take $O(I_{\max} \cdot k \cdot 3^k \cdot N + 2^k \cdot N \log N)$ time, excluding the greedy start and ILP solves. □

A.5 Prefix-Ready Admission

At runtime, a segment is ready if its KV states are cached, in flight if an admitted producer has claimed it, and pending otherwise. The dependency property below explains why admission can decide whether to hold a waiting request by checking only its first unready segment, rather than every segment on its path individually.

Lemma 8 (Segment dependency). *For a waiting request, prefill can start at segment s only if all preceding segments on its path are ready. If a segment is pending, all its descendants are pending.*

Proof of Lemma 8. Prefill computes a prompt from left to right, and the radix cache [14] evicts a node only after its descendants, so a cached prefix retains all preceding tokens. For a newly admitted request to compute s while reusing all preceding segments, their KV states must be available in the cache, so those segments must be ready. An already running producer can instead use its own computed KV states for its subsequent prefill steps.

For the second statement, suppose that s is not pending. If s is ready, cache prefix closure makes all preceding segments ready. If s is in flight, its admitted producer has claimed every uncached segment that it will compute, including any uncached predecessors of s; cached predecessors are ready. Thus every preceding segment is ready or in flight, even if the producer has not yet computed it. No nonpending segment has a pending predecessor. By contraposition, every descendant of a pending segment must also be pending. □

B DP FOR SMALL SCOPES

The DP relies on the fact that any order of the evidence items is allowed, as in our problem. If some items of a request had to keep a given order, the DP would no longer apply, whereas the ILP of Eq. (1) would still apply after removing the orders that break this requirement from $O(r)$. The ILP can then optimize prefix sharing over these legal orders without changing its objective or weights.

This section gives a solver-free DP alternative for the small scopes of Section 4.1, without dependencies. It reaches the same minimum as the ILP of Eq. (1) without a solver and within a predictable time. Since a group may contain smaller groups, a request can belong to several nested groups. Without dependencies, the request places the evidence shared by these groups at the front, from the largest group to the smallest, followed by its remaining evidence. The order of every request thus follows from the groups it belongs to, so DP only decides how the requests nest into groups. With dependencies, a shared evidence may have to wait for an item that only some members retrieve, so the ILP is needed instead.

Let $I(G)$ be the evidence shared by all requests in G, and let $L(G) = \sum_{e\in I(G)} \ell(e)$. Candidate groups are unions of classes with identical evidence sets, and $|G|$ counts requests, including their multiplicities. If G lies directly within a larger group U, it shares $L(G) - L(U)$ tokens beyond U and computes them once rather than $|G|$ times, which saves $s(G,U) = (|G| - 1)\big(L(G) - L(U)\big)$ tokens. Minimizing W thus becomes maximizing the total saving. Let $Save(U)$ denote the largest total saving that the groups within U can obtain. DP computes it over disjoint eligible subgroups:

$$Save(U) = \max_{G_1,\dots,G_m} \sum_{j=1}^{m} \Big(s(G_j, U) + Save(G_j) \Big), \tag{7}$$

where $G_1, \dots, G_m$ are disjoint proper subgroups of U, each with at least two requests and $L(G_j) > L(U)$; choosing no group gives 0. For a scope S, DP starts at a separate virtual root $\perp$ with $I(\perp) = \emptyset$ and $L(\perp) = 0$. At this root, groups may be any subsets of S, including S itself. DP backtracks the maximizing choices into nested groups.

Proposition 4 (Optimality of DP). *For a scope with d distinct evidence sets and no dependencies, DP finds a layout of minimum weight. Its recurrence takes $O(4^d)$ time after the intersection lengths and request multiplicities of candidate groups are precomputed.*

Appendix A.2 proves Proposition 4 by matching the recurrence's saving to a lower bound on the weight of every legal layout, and then bounding the time required to enumerate its subgroup choices.

C ADDITIONAL EXPERIMENT

C.1 Sensitivity to the Number of Evidence Items

(RQ13) Do the evidence-count trends hold across all four datasets? To further evaluate sensitivity to evidence count, we conducted a sweep on all four datasets with $k = 3$–6 evidence items per request, Llama-3.1-8B, and an 8 GiB KV cache budget per GPU. This extends RQ12 beyond DragonBall and WikiCities to examine whether the same trends hold on NarrativeQA and SQuAD. Each public-dataset question retrieves its top-k chunks with BM25, while each of the 3,000 WikiCities requests compares k cities. All systems of a dataset run on the same pair of GPUs. We exclude BatchLLM because it uses its own engine. Throughput includes both planning



Figure 1: End-to-end throughput vs. evidence count k (Llama-3.1-8B, 8 GiB KV cache per GPU).

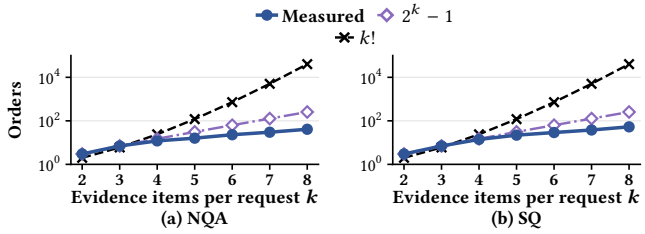


Figure 2: Orders that the index distinguishes per request.

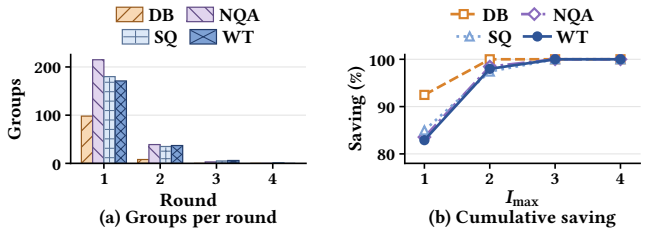


Figure 3: Effect of the number of rounds I_{max}.

and inference, so the comparison accounts for the cost of finding the evidence orders as well as executing the resulting layouts.

Figure 1 shows that KaleidoKV has the highest throughput in all 16 settings. Its lead over the fastest baseline decreases from 1.52–2.01× at $k = 3$ to 1.14–1.36× at $k = 6$. Two observations explain this narrowing gap. First, longer prompts reduce request concurrency, so fewer requests begin prefill before their shared prefixes are cached. ContextPilot, GGR, and SOLO compute 24%–73% more prefill than they plan at $k = 3$, but only 1.9%–23% more at $k = 6$. KaleidoKV stays within 1.8% of its plan throughout the sweep. Second, ContextPilot's planned prefill approaches ours on DragonBall, NarrativeQA, and SQuAD, from 5.2%–12.0% above ours at $k = 3$ to 1.3%–6.4% at $k = 6$. On WikiCities, it remains 17.5%–21.2% above ours, and the throughput lead remains the largest there. Thus, a smaller throughput advantage can reflect both less duplicate execution and closer layout quality, even while KaleidoKV continues to realize nearly all planned reuse across the tested evidence counts.

Planning time grows from 6.4–7.8 s at $k = 3$ to 28.7–33.5 s at $k = 6$, but remains below 8% of end-to-end time. Inference remains the dominant cost across these evidence counts. Together, the results show that the advantage persists across all four datasets, while its magnitude depends on both the sharing opportunities in the workload and the concurrency allowed by longer prompts under the same per-GPU cache budget for every compared system.

C.2 Planner and Workload Sensitivity

(RQ14) How many orders does the index distinguish as requests retrieve more evidence? To characterize how evidence count affects candidate orders, we conducted a sweep on NarrativeQA and SQuAD with $k = 2$–8, counting classes of reused evidence sets per request in the final layout (Figure 2). These classes distinguish what an order reuses, rather than every permutation of its evidence items. By Lemma 7, orders that reuse the same set have the same gain, and there are at most 2^k such classes, including the empty reused set. The plotted $2^k - 1$ reference counts nonempty subsets, while $k!$ gives the number of full evidence permutations.

The measured counts grow much more slowly than either reference. At $k = 8$, the maximum is 41 on NarrativeQA and 53 on SQuAD, while the means are 10.3 and 13.9. These counts are far below the 40,320 possible permutations and below the bound on reused sets. The distinction matters because different suffix orders need not be evaluated separately when they expose the same reusable prefix items. The index can reason over these reuse classes when choosing an order. This experiment supports the reduction in candidate evaluation on the tested layouts, complementing the planning-time and batch-scaling results in RQ6 and RQ8.

(RQ15) How many rounds does regrouping need? To examine sensitivity to the regrouping limit in joint layout planning, we conducted a round-limit sweep on all four datasets with $I_{max} = 1$–4, comparing the resulting layouts with the default $I_{max} = 10$ (Figure 3). We examine both how many groups are dissolved in each round and how much of the total saving has been captured. These measurements distinguish the number of revisions still being made from their contribution to reducing the prefill cost of the planned layout after each round of merge and dissolve operations.

The first round dissolves 98–215 groups and captures 83%–92% of the total saving. Later rounds affect fewer groups, and the tested layouts stop changing within four rounds, matching the default. Most of the benefit is thus obtained early, while subsequent rounds refine a smaller set of groups. Restricting the planner to one round changes planned prefill by at most 0.23%, compared with up to 0.65% variation across repeated runs, and changes throughput by at most 0.8%. The small differences show that the end-to-end gain does not depend on using all ten rounds in these experiments. The default leaves room for further refinement, while early stopping avoids additional rounds once a round dissolves no group.

(RQ16) How does the density of evidence sharing affect the lead of KaleidoKV? To examine how the advantage of KaleidoKV depends on the structure of the workload, we conducted a sweep on WikiCities with pools of 48, 32, and 16 cities. The pool size controls the density of sharing, since a smaller pool makes more requests draw the same pages. Each workload uses the same generator and seed as Appendix D, with 2,000 requests of four cities each, and we use Llama-3.1-8B with a KV cache budget of 4 GiB per GPU. All

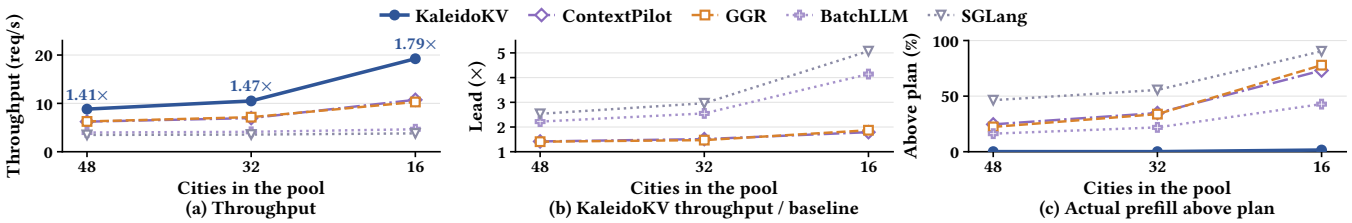


Figure 4: Effect of sharing density on WikiCities. Fewer cities make sharing denser. Labels in (a) give the lead over the fastest baseline. (b) reports KaleidoKV's throughput relative to each baseline, and (c) measures actual prefill above the plan.

systems run on the same pair of GPUs, and SOLO was not part of this sweep. As the pool shrinks from 48 to 16 cities, the fraction of the batch sharing a page with a request rises from 70.9% to 92.3%, while distinct evidence sets in each batch fall from 1,700 to 632.

Figure 4(a) shows that KaleidoKV has the highest throughput at every pool size, and Figure 4(b) shows that its lead grows as sharing becomes denser. The lead over the fastest baseline rises from 1.41× at 48 cities to 1.47× at 32 cities and 1.79× at 16 cities. Over the same range, the lead rises from 2.22× to 4.14× over BatchLLM and from 2.53× to 5.07× over SGLang. The throughput of KaleidoKV grows by 2.18×, compared with 1.73× for ContextPilot and 1.64× for GGR.

Two observations explain this trend. First, denser sharing increases the prefill that the baselines repeat during execution, which is consistent with more requests overlapping a prefix that is still being computed. Figure 4(c) shows that the actual prefill of ContextPilot exceeds its plan by 24.6% at 48 cities and by 73.1% at 16 cities, and the excess of GGR grows from 22.3% to 77.9%. In contrast, KaleidoKV stays within 1.6% of its plan, since prefix-ready admission holds these requests until the shared prefix is ready. Second, the planned prefill of ContextPilot grows from 16.8% to 20.9% above ours, and that of GGR grows from 6.7% to 10.5% above ours, because coordinated grouping has more overlapping requests to place on shared prefixes. As a result, the actual prefill of ContextPilot grows from 1.45× to 2.06× ours. These results suggest that the lead of KaleidoKV is largest on workloads in which many requests reuse a small set of evidence in different combinations across the batch.

D EXPERIMENTAL SETUP

Datasets and Workload Construction. Following [4, 9], we use three public question-answering datasets and one synthetic workload. We use the same abbreviations as in Section 5.1: DragonBall (DB), NarrativeQA (NQA), SQuAD (SQ), and WikiCities (WT).

- **DragonBall (DB)**. DragonBall [16] is a RAG benchmark over finance, law, and medical documents. We use all 3,108 English questions, each retrieving documents only from its own domain.
- **NarrativeQA (NQA)**. NarrativeQA [6] contains free-form questions about books and movie scripts. We use all 3,461 validation questions of the summary setting, with retrieval over the whole validation split rather than a separate evidence pool for each question, with BM25 ranking evidence chunks separately for each request.
- **SQuAD (SQ)**. SQuAD [10] contains extractive questions about Wikipedia articles. We select whole development articles in title order until the question count reaches at least 3,000, yielding 3,080 questions. Retrieval uses the whole selected development split.

Evidence preparation and sharing. For the three public datasets, we merge consecutive paragraphs of each document into chunks of at least 380 words, with each chunk serving as one evidence item. By default, each question retrieves four chunks with BM25, and retrieval rank determines their original order. We retain every question in the selected batches. A request shares a chunk with 5.3% of the DB batch, 9.4% of NQA, and 5.6% of SQ on average. These datasets capture localized sharing, where only a small fraction of a batch has any retrieved evidence in common with a given request.

- **WikiCities (WT)**. Offline batches may also reuse a small pool of popular evidence in many combinations, as in analytical queries over an entity catalog [8, 9]. To study this pattern, we construct WikiCities from the 32 most populous cities with complete facts. Each city's page combines its Wikipedia introduction with its Wikidata facts. Each request asks a comparison question [2, 11] about four cities, such as which has the largest population, and uses their pages as evidence. Thus, the selected city pages directly determine the evidence set rather than a BM25 retrieval step over all pages.

WikiCities Construction. We sample cities from a Zipf(1) distribution over population rank, motivated by the popularity skew of chunks in RAG retrievals [1]. The draw order determines the original evidence order. We discard ties and independently shuffle the cities in the question, so neither order reveals the answer. Using a fixed seed, we generate 3,000 requests with 1,889 distinct page sets. A request shares a page with 79% of the batch on average, exposing reuse opportunities across many overlapping request groups.

Baseline Configurations. We compare KaleidoKV with five systems. ContextPilot, GGR, and SOLO reorder evidence to create shared prefixes. BatchLLM and SGLang retain the original evidence order and reuse existing shared prefixes in the input prompts.

- **ContextPilot**. ContextPilot [4] clusters requests by the pairwise distance of their evidence, aligns evidence within each cluster into shared prefixes, and schedules similar requests together. We use its offline mode, which builds the context index before the batch starts. Disabling annotation isolates alignment and reordering.
- **GGR**. GGR [9] uses a recursive greedy algorithm to reorder the fields within each row and the rows of a table to maximize prefix hits. We use the official implementation, representing each request as a row and its evidence chunks as the fields of that row.
- **SOLO**. SOLO [8] greedily chooses one field order for all rows and then sorts the rows. We implement its published algorithm and treat each distinct chunk as a field. The resulting chunk order is shared across requests, unlike request-specific evidence orders.

• **BatchLLM**. BatchLLM [15] extracts common prefixes from a batch and computes each once before the requests that use it. We use its public implementation based on vLLM [7] 0.6.4 (`microsoft/MixLLM`, branch `batchllm_vllm_064`), with global prefix clustering, chunked prefill, and token batching enabled. The paper delegates prefixes below the first level to vLLM's LRU cache, but this implementation disables prefix caching when global prefix clustering is enabled. Our BatchLLM runs thus have no LRU cache. As reported in Section 5.1, it does not support Qwen3-8B and fails on Qwen2.5-14B-Instruct with NQA. Table 2 marks these unsupported configurations and failed runs with dashes.

• **SGLang**. SGLang [14] serves requests in their original evidence order using its default first-come-first-served scheduling and RadixAttention prefix caching. Its longest-prefix-match policy falls back to first-come-first-served when more than 128 requests are waiting. This condition holds for about 96% of scheduling steps on our batches. The throughput of this alternative policy stays within 1% of the default across all 16 evaluated dataset–model settings.

Comparison scope. We exclude methods such as CacheBlend [12] that reuse KV states of non-prefix chunks. These methods recompute only part of the reused states and may change model outputs. Our comparison focuses on prefix-based reuse, which reuses KV states only when the preceding tokens match the current prompt.

Evaluation Metrics. We report efficiency, prefill work, and answer quality using the abbreviations in the main tables and figures.

• **Throughput (Thr.)**. Throughput is the number of requests divided by the batch's end-to-end runtime, measured in requests per second. The runtime includes planning but excludes model loading. It measures the combined cost of preparing evidence orders and executing all requests, accounting for the planning overhead when comparing the end-to-end benefit of shared-prefix layouts.

• **Time to first token (TTFT)**. For each request, TTFT is measured from batch submission to its first output token, including planning and waiting before execution. We average TTFT across the batch and report each system's mean relative to KaleidoKV. A ratio above 1.00× indicates a longer mean wait for the first token than KaleidoKV under the same dataset and model configuration.

• **Planned and actual prefill**. Planned prefill (Plan.) counts prompt tokens assuming that each shared prefix is computed once. Actual prefill counts the tokens computed by the engine, including repeated computation of a shared prefix. Both cover complete prompts: instruction, evidence, and question. RQ5 uses the evidence-level weight W instead, comparing it with the ILP lower bound from Eq. (1). This distinction separates the evidence-order optimization objective from full-prompt execution work measured by the engine.

• **Answer quality (Qual.)**. We use ROUGE-L for DB and NQA, normalized exact match (%) for SQ, and position-balanced accuracy (%) for WT. The WT metric accounts for a model's possible preference for the first page. It groups requests by the position of the answer's page and weights each group's accuracy by its size under the original evidence order. Quality comparisons use the same dataset-specific metric across systems, rather than comparing scores from different datasets as if they measured the same quantity across QA tasks with different answer formats and metrics.

Models and Environment. We use four open instruction-tuned models, each served with tensor parallelism over two GPUs.

Models. We use Qwen2.5-14B-Instruct, Qwen3-8B, Llama-3-8B-Instruct, and Llama-3.1-8B-Instruct. They cover two model sizes and two model families, with two versions from each family. All run in BF16, and Qwen3-8B runs with thinking disabled.

Hardware and software. The server has two Intel Xeon Silver 4314 CPUs (32 cores in total), 503 GB of host memory, and NVIDIA RTX A5000 GPUs with 24 GB of memory each. It runs Ubuntu 22.04 and CUDA 12.8. For each dataset and model, all systems use the same pair of GPUs, and a guard aborts a run if another process uses those GPUs during any part of the measured run, to avoid interference.

Implementation and Runtime Settings. Unless an experiment states otherwise, we use the following serving engines, inference limits, prompts, and planner settings, together with the baseline-specific options described above for each corresponding system.

Serving engines. All systems except BatchLLM run on SGLang 0.5.4 with FlashInfer attention [13]. BatchLLM uses its own engine based on vLLM 0.6.4. Accordingly, RQ2 and Appendix C.1 exclude BatchLLM when comparing cache budgets and evidence counts on a shared serving engine with a common attention backend.

Shared inference settings. All systems use chunked prefill with 2,048 tokens per chunk, at most 256 concurrent requests, greedy decoding with at most 32 output tokens, and a context window of 8,192 tokens. The default KV cache budget is 4 GiB per GPU, checked at the start and end of every run. Cache-capacity experiments state each tested budget per GPU, allowing their results to be distinguished from the default 4 GiB setting used in the main comparison across all four datasets and four models in Table 2.

Requests and prompts. Systems receive the same requests, evidence, and prompt template. Following [4], each prompt begins with a fixed instruction, followed by the evidence chunks and then the question. Only the evidence order changes, not the evidence set, its content, or the surrounding instruction and the question text.

Planner configuration. KaleidoKV treats a scope as small when it has at most $\theta = 15$ distinct evidence sets and solves its ILP with HiGHS [3]. We set $I_{\max} = 10$ for Dissolve. This limit never binds in the experiments, since Dissolve stops within 8 passes on every dataset and model. The implementation selects at most $n_{\max} = 150$ requests per Merge candidate. The complexity analysis instead uses uncapped candidate lists and their first-item partitions for Merge.

Request order and admission. Requests enter the engine in the depth-first order of the planned prefix tree, visiting each node's subtrees in increasing order of their tokens per request. Prefix-ready admission scans the first $w = 64$ waiting requests at each engine step. Because chunked prefill spans multiple steps, an admitted request claims every segment it will compute. Those segments remain in flight until their KV states are cached. Later requests can distinguish in-flight computation from cached prefixes and wait for its producer, avoiding duplicate prefill of shared segments.

A request whose overlap with in-flight segments is shorter than $\tau = 32$ tokens is admitted directly, since recomputing such a short overlap is cheaper than waiting for it. If all scanned requests are held, the engine admits requests in its own order. Neither fallback changes the planned evidence orders or prompt contents.

REFERENCES

[1] Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–28.

[2] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. 6609–6625.

[3] QHJAJ Huangfu and JA Julian Hall. 2018. Parallelizing the dual revised simplex method. *Mathematical programming computation* 10, 1 (2018), 119–142.

[4] Yinsicheng Jiang, Yeqi Huang, Liang Cheng, Cheng Deng, Xuan Sun, and Luo Mai. 2026. ContextPilot: Fast Long-Context Inference via Context Reuse. *Proceedings of Machine Learning and Systems* 8 (2026), 1639–1655.

[5] Richard M Karp. 1972. Reducibility among combinatorial problems. In *Complexity of Computer Computations: Proceedings of a symposium on the Complexity of Computer Computations*. Springer, 85–103.

[6] Tomáš Kočiskỳ, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. 2018. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics* 6 (2018), 317–328.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

[8] Yingze Li, Dong Wang, Yiming Guo, Yao Chen, Hongzhi Wang, and Bingsheng He. 2026. Prefix-Cache-Aware Data Reordering for LLM-Augmented Database Analytics. In *Proceedings of the 43rd International Conference on Machine Learning (PMLR)*, Vol. 306.

[9] Shu Liu, Asim Biswal, Amog Kamsetty, Audrey Cheng, Luis G Schroeder, Liana Patel, Shiyi Cao, Xiangxi Mo, Ion Stoica, Joseph E Gonzalez, et al. 2025. Optimizing llm queries in relational data analytics workloads. *Proceedings of Machine Learning and Systems* 7 (2025).

[10] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*. 2383–2392.

[11] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*. 2369–2380.

[12] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. Cacheblend: Fast large language model serving for rag with cached knowledge fusion. In *Proceedings of the twentieth European conference on computer systems*. 94–109.

[13] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, et al. 2025. Flashinfer: Efficient and customizable attention engine for llm inference serving. *Proceedings of Machine Learning and Systems* 7 (2025).

[14] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* 37 (2024), 62557–62583.

[15] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. 2026. Batchllm: Optimizing large batched llm inference with global prefix sharing and throughput-oriented token batching. *Proceedings of Machine Learning and Systems* 8 (2026), 1742–1756.

[16] Kunlun Zhu, Yifan Luo, Dingling Xu, Yukun Yan, Zhenghao Liu, Shi Yu, Ruobing Wang, Shuo Wang, Yishan Li, Nan Zhang, et al. 2025. Rageval: Scenario specific rag evaluation dataset generation framework. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8520–8544.